

코드가 아니라 결과를 검증하는 사람입니다.

01 / ABOUT

자기소개

AI 기반 소프트웨어 품질관리 과정을 통해 **Python, AI QA, LLM 평가, 웹 개발**을 학습하며 실습 중심의 프로젝트를 수행했습니다. 단순히 강의를 수강하는 것에 그치지 않고, 직접 프로젝트를 구현하고 테스트하며 **AI 서비스 품질을 평가하는 경험**을 쌓았습니다. 4인 팀 RAG 챗봇 품질평가 프로젝트에서는 발표를 맡으며 전체 QA 프로세스를 이해하고 전달하는 경험을 했고, 이후 개인적으로 Judge Agent와 Correction Agent를 직접 구현하며 더 깊게 파고들었습니다.

● PROJECT_01 · RAG QA · PASS

● PROJECT_02 · BERTScore · PASS

● PROJECT_03 · 회귀평가 · PASS

● PROJECT_04 · Routing · PASS

● PROJECT_05 · WEB · PASS

02 / PROJECTS

프로젝트

PROJECT 01 · FEATURED

RAG 챗봇 자동 품질평가 & 답변 개선 시스템

Judge Agent로 채점하고, Correction Agent로 실패 답변을 자동 개선하는 QA 피드백 루프

Python

LangChain

OpenAI GPT-4.1-mini

ChromaDB

Streamlit

RAG 기반 문서 챗봇

등록된 문서 내용에 근거하여 답변합니다.

PROBLEM — 문제

RAG 챗봇은 검색된 문서를 근거로 답변하지만, 검색이 잘 됐는지·답변이 근거에 맞는지·문서에 없는 내용을 지어내지(환각) 않는지를 사람이 매번 직접 확인하면 기준이 흔들리고 시간이 오래 걸립니다. 또한 점수만 매겨서는 낮은 점수를 받은 답변이 그대로 폐기되어, 평가가 실제 개선으로 이어지지 않는다는 한계가 있었습니다.

PROCESS — 해결 과정

- RAG 파이프라인 구축** — 문서 임베딩 → ChromaDB 저장 → 질문 시 유사 문서 검색 → 근거 기반 답변 생성
- 테스트 케이스 설계** — 정상·경계·환각 유도·안전 거절 등 15개 카테고리로 시나리오 구성
- Judge Agent 구현 (1단계)** — 이해도·정확성·관련성·표현성 4항목을 0~5점 루브릭으로 채점
- 감점 규칙 적용 (2단계)** — 출처 미기재·근거 미사용 등 위반 시 정량 감점 → 최종점수 = 루브릭 - 감점
- Correction Agent 구현** — 평가 리포트(JSON)를 입력받아 감점 원인을 반영한 수정 모범답안 자동 생성
- 리포트 자동화** — 전체 결과를 JSON + Markdown 리포트로 자동 저장

WHY TWO-STAGE — 평가를 "루브릭(잘한 점)"과 "감점(어긴 규칙)"으로 분리한 이유는 단일 점수만 주면 근거가 불투명해지고 점수가 후하게 매겨지는 인플레이션이 생기기 쉽기 때문입니다. 분리하면 "왜 이 점수인지"를 항상 설명할 수 있고, 감점 사유를 그대로 Correction Agent에 넘겨 개선까지 자동으로 이어집니다.

RESULT — 결과

4.40 / 5.00

15

5.00

2.75

전체 평균 (15개 케이스)

테스트 카테고리

최고 점수

최저→자동개선

BEFORE · 2.75/5.00

Q. "RAG 챗봇이 무엇인지 쉽게 설명해 주세요."

"제공된 문서에서는 확인할 수 없습니다."

→ 감점: 질문 의도 파악 실패, 출처 미언급



AFTER · Correction Agent

"RAG 챗봇은 'Retrieval-Augmented Generation'의 약자로, 외부 문서에서 정보를 검색한 후 이를 바탕으로 답변을 생성하는 챗봇입니다. 참고로 제공된 문서에는 RAG 챗봇에 대한 내용이 포함되어 있지 않습니다."

→ 개선: 직접 답변 + 출처 부재 명시

- ✓ **Two-Stage Evaluation**으로 평가 점수의 근거를 항상 설명 가능하게 설계했습니다.
- ✓ 평가에서 멈추지 않고 **개선까지 이어지는 피드백 루프**를 구현했습니다.
- ✓ Prompt Engineering, RAG, Vector DB를 함께 다루며 **AI 서비스 품질관리 자동화**를 경험했습니다.

PROJECT 02

BERTScore 기반 AI 답변 품질 평가 시스템

의미 기반 유사도로 챗봇 응답 품질을 자동 채점하는 QA 파이프라인

Python

bert-score

PyTorch

pandas

PROBLEM — 문제

교육기관 챗봇의 답변 품질을 사람이 매번 눈으로 확인하면 비효율적이고, 단순 문자열 비교(키워드 일치)는 문장 표현이 다른 의미가 같아도 오답으로 판정하는 한계가 있었습니다. **의미 기반으로 정답 여부를 자동 판단**할 평가 체계가 필요했습니다.

PROCESS — 해결 과정

- 1 **Golden Answer 설계** — 정답 기준이 되는 모범 답안을 케이스별로 작성
- 2 **테스트 데이터 구조화** — test_cases.json / model_answers.json 으로 입력 분리
- 3 **BERTScore 계산** — Golden Answer와 AI 답변(Model A, B) 간 의미적 유사도 산출
- 4 **Winner 자동 판정** — 두 모델 점수를 비교해 케이스별 우승 모델 자동 기록
- 5 **CSV 저장 & 평균 분석** — pandas로 결과를 누적·집계해 전체 성능 비교

RESULT — 결과

CASE	MODEL A	MODEL B	WINNER
Q001	0.9812	0.6804	Model A
Q002	0.9321	0.7102	Model A

- ✓ 문자열 비교가 아닌 **의미 기반 평가**의 필요성과 원리를 이해했습니다.
- ✓ **Golden Answer 설계**가 평가 신뢰도를 좌우한다는 점을 학습했습니다.
- ✓ Python 기반 **AI QA 자동화 파이프라인** 구축 경험을 쌓았습니다.

PROJECT 03

기준 모델 기반 BERTScore 회귀 평가

새 모델(B)이 기준 모델(A)의 응답 특성을 유지하는지 배포 전에 자동 점검하는 회귀 테스트

Python

bert-score

pandas

PROBLEM — 문제

Golden Answer 기반 평가는 "정답에 가까운가"는 보여주지만, 모델을 교체했을 때 기존 서비스 품질이 그대로 유지되는지는 별도로 확인해야 합니다. 새 모델이 더 효율적이거나 저렴하더라도, 기존 모델과 응답 방향이 달라지면 사용자 경험이 갑자기 바뀔 수 있기 때문입니다.

PROCESS — 해결 과정

- 1 **기준 모델(A) 응답을 Reference로 설정** — 정답이 아니라 "기존 서비스가 보이던 답변"을 기준으로 사용
- 2 **비교 모델(B) 응답과 BERTScore 산출** — Precision/Recall/F1로 두 응답의 의미적 거리 측정
- 3 **3단계 자동 판정 기준 설계** — F1 0.95 ↑ 안정적 유지 / 0.85 ↑ 검토 필요 / 0.85 ↓ 성능 저하 의심
- 4 **케이스별 평가 의견 자동 생성** — 판정 결과에 맞는 QA 코멘트를 템플릿으로 자동 작성
- 5 **종합 보고서 자동화** — 전체 평균 F1, 판정 분포, 배포 권고 사항을 Markdown으로 출력

RESULT — 결과

0.797 평균 F1

0

1

9

안정적 유지

검토 필요

성능 저하 의심

10개 케이스 (TC-001~010)

Model A (기준) · TC-006

Q. "BERTScore는 무엇을 평가하는 지표인가요?"

"BERTScore는 두 문장의 의미적 유사도를 측정하는 평가 지표입니다."



Model B · F1 0.802 · 성능 저하 의심

"BERTScore는 문장 간 의미가 얼마나 비슷한지 비교하는데 사용됩니다."

→ 의미는 거의 동일하지만 표현 차이만으로 "저하 의심" 분류

- ✓ BERTScore는 **표현 차이에 민감**해서, 의미가 같아도 점수가 낮게 나올 수 있다는 한계를 확인했습니다.
- ✓ 그래서 **자동 점수는 1차 필터일 뿐, 최종 판단은 사람의 검토와 결합**해야 한다는 QA 원칙을 정리했습니다.
- ✓ 안전성·환각·숫자·금액처럼 **자동 지표만으로 판단하면 안 되는 영역**을 구분하는 기준을 세웠습니다.

PROJECT 04

Routing Classifier 기반 AI Agent 자동 분류 품질평가

사용자 질문을 올바른 전문 Agent로 연결하는 분류기의 정확도와 오분류 원인을 분석

Python

scikit-learn

TF-IDF

Logistic Regression

matplotlib

PROBLEM — 문제

여러 전문 Agent(스포츠/타이머/주문/일반대화)를 가진 챗봇에서는, 질문에 잘 답하는 것보다 올바른 Agent로 먼저 연결되는 것이 우선입니다. "결제 방법 알려줘"가 스포츠 Agent로 잘못 연결되면, 답변 품질과 무관하게 사용자는 원하는 도움을 받을 수 없습니다.

PROCESS — 해결 과정

- 1 **분류기 학습** — TF-IDF(문자 2~5-gram) + Logistic Regression으로 4개 Agent 분류 파이프라인 구축
- 2 **정상 케이스 평가** — 스포츠/타이머/주문/일반대화 16건으로 Accuracy, Precision/Recall/F1 측정
- 3 **혼동행렬(Confusion Matrix) 분석** — 어떤 Agent끼리 혼동되는지 시각화
- 4 **Edge Case 설계** — 오타, 복합 의도, 의미 없는 입력, 짧은 문장 등 8건 별도 테스트
- 5 **오분류 원인 분석 & 개선안 도출** — 데이터 불균형과 문맥 부족 문제를 구체적 개선 방향으로 정리

RESULT — 결과

93.75%

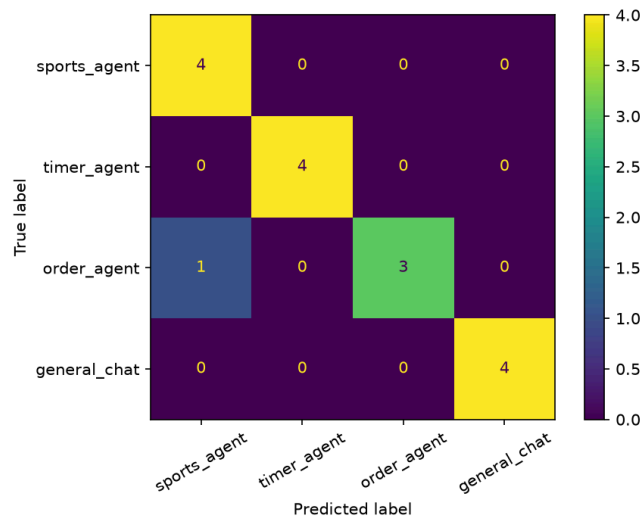
정상 케이스 정확도 (15/16)

87.5%

Edge Case 정확도

4

분류 Agent 수



오분류 사례

"결제 방법 알려줘"

실제: order_agent → 예측: sports_agent

→ 짧은 문장 + order_agent 학습 데이터 부족

개선 방안

결제·환불·배송 표현 다양화, 오타·복합 의도 데이터 추가, Sentence-BERT 기반 임베딩 분류기와 성능 비교 실험

- ✓ Accuracy 하나만 보지 않고 **혼동행렬로 어떤 Agent 간 혼동이 발생하는지** 분석하는 습관을 들였습니다.
- ✓ 정상 케이스만으로는 부족하며 **오타·복합의도 등 Edge Case 테스트가 따로 필요**하다는 것을 확인했습니다.
- ✓ 모델 성능 평가는 Accuracy뿐 아니라 **Precision/Recall/F1과 오분류 사례 분석이 함께** 이루어져야 함을 체득했습니다.

PROJECT 05

모바일 청첩장 웹페이지 제작

반응형 UI와 인터랙션을 직접 설계·구현한 모바일 퍼스트 웹페이지

HTML5

CSS3

JavaScript

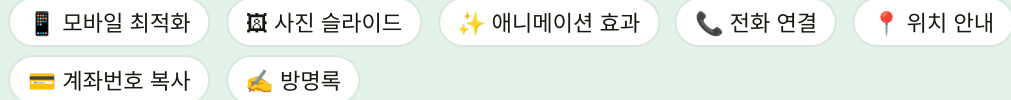
PROBLEM — 문제

모바일 청첩장은 대부분 스마트폰으로 열람되기 때문에, 화면 크기·터치 인터랙션을 고려하지 않으면 사진이 깨지거나 버튼을 누르기 어려운 등 사용성이 떨어집니다. 전화 연결, 위치 안내, 계좌번호 복사처럼 모바일 환경에 특화된 기능도 함께 필요했습니다.

PROCESS — 해결 과정

- 1 **프로젝트 구조 설계** — 페이지 섹션과 폴더 구조를 먼저 설계
- 2 **HTML/CSS 구현** — 시맨틱 마크업과 모바일 우선 반응형 스타일링
- 3 **JavaScript 인터랙션** — 사진 슬라이드, 스크롤 애니메이션 효과 구현
- 4 **핵심 기능 추가** — 전화 연결, 위치 안내, 계좌번호 복사, 방명록 기능 구현
- 5 **테스트** — 다양한 화면 크기에서 레이아웃과 기능 동작 확인

RESULT — 결과



- ✓ 모바일 환경을 중심에 둔 **반응형 UI**를 직접 구현했습니다.
- ✓ 사용자 경험(UX)을 고려한 **화면 설계** 경험을 쌓았습니다.
- ✓ 기획부터 구현까지 이어지는 **프로젝트 구조화** 경험을 얻었습니다.

03 / SKILLS

보유 기술

Python

- 함수 작성
- JSON 처리
- pathlib
- pandas
- 파일 입출력

AI QA

- BERTScore
- Embedding
- Word2Vec
- Cosine Similarity
- Precision/Recall/F1
- Golden Answer 설계
- LLM-as-a-Judge
- Two-Stage Evaluation
- RAG · LangChain · ChromaDB
- 회귀 테스트(Regression Test)
- scikit-learn · Confusion Matrix

Web

- HTML5
- CSS3
- JavaScript
- 반응형 웹

04 / GOAL

앞으로의 목표

LLM 기반 AI 서비스를 테스트하고 품질을 평가할 수 있는 **AI QA 엔지니어**로 성장하는 것을 목표로 합니다. Python 자동화와 AI 평가 기술을 지속적으로 학습하여 실제 서비스 품질 향상에 기여하는 QA 엔지니어가 되고자 합니다.